

Claude Code en Acción. Descripción general del curso

○¿Qué es el Código Claude?

Introducción

¿Qué es un asistente de codificación/programación?

Claude Code en acción

○Ponerse manos a la obra

Configuración del código Claude

Configuración del proyecto

Añadiendo contexto

Realizar cambios

Encuesta de satisfacción con el curso

Contexto de control

Comandos personalizados

Servidores MCP con código Claude

Integración con GitHub

○Ganchos y el SDK

Presentando los ganchos

Definiendo ganchos

Implementar un gancho

Trampas alrededor de los anzuelos

¡Ganchos muy útiles!

Otro gancho útil

El SDK de Claude Code

○Para finalizar

Cuestionario sobre el Código Claude

Resumen y próximos pasos

Acerca de este curso

Configuración del código Claude

¡Es hora de configurar Claude Code localmente!

Puedes encontrar las instrucciones completas de configuración aquí : <https://code.claude.com/docs/es/quickstart>

En resumen, deberá hacer lo siguiente :

1. Install Claude Code

- a. macOS, Linux, WSL : `curl -fsSL https://claude.ai/install.sh | bash`
- b. Windows PowerShell : `irm https://claude.ai/install.ps1 | iex`
- c. Símbolo del sistema de Windows (cmd.exe) : `curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd`
- d. MacOS (Homebrew) : `brew install --cask claude-code`

2. **Tras la instalación, ejecútalo Claude en tu terminal.** La primera vez que ejecutes este comando, se te pedirá que elijas un tema de color para la terminal y que te autentiques con tus credenciales de claude.ai.

Si después de la instalación aparece un error que Claude no se encuentra en la lista de problemas detectados, o si se produce un error de red o de permisos, consulte la sección "[Solucionar problemas de instalación](#)" en la documentación.

¿Utilizas Claude Code a través de Amazon Bedrock, Google Cloud Vertex AI o Microsoft Foundry? Consulta [la configuración del proveedor externo](#) para obtener instrucciones adicionales.

<https://code.claude.com/docs/es/quickstart>

Comandos esenciales

Aquí están los comandos más importantes para el uso diario :

Comando	Qué hace	Ejemplo
claude	Inicia el modo interactivo	claude
claude "task"	Ejecuta una tarea única	claude "fix the build error"
claude -p "query"	Ejecuta una consulta única y luego sale	claude -p "explain this function"
claude -c	Continúa la conversación más reciente en el directorio actual	claude -c
claude -r	Reanuda una conversación anterior	claude -r
/clear	Borra el historial de conversación	/clear
/help	Muestra los comandos disponibles	/help
exit o Ctrl+D	Salir de Claude Code	exit

Configuración del proyecto

Trabajar con Claude Code resulta más interesante si se dispone de un proyecto en el que trabajar.

He creado un pequeño proyecto para explorar con Claude Code. Es la misma aplicación de generación de interfaz de usuario que mostré en un video anterior. Nota : no es necesario ejecutar este proyecto. ¡Si lo deseas, puedes seguir el resto del curso con tu propio código!

Configuración

Este proyecto requiere una pequeña cantidad de configuración :

1. **Asegúrese de tener Node JS instalado localmente.** [Enlace a las instrucciones de instalación](#) .
2. **Descarga el archivo zip adjunto uigen.zip** a esta clase y extráelo.
3. **En el directorio del proyecto, ejecute el comando `npm run setup`** para instalar las dependencias y configurar una base de datos SQLite local.
4. **Opcional :** este proyecto utiliza Claude a través de la API de Anthropic para generar componentes de interfaz de usuario. Si desea probar la aplicación por completo, deberá proporcionar una clave API para acceder a la API de Anthropic. *Puede omitir este paso; la aplicación seguirá generando código estático ficticio.* A continuación, se explica cómo configurar la clave API :
 - a. Obtén una clave API de Anthropic en <https://console.anthropic.com/>
 - b. Introduce tu clave API en el `.env` archivo. Sustituye el texto literal `your-api-key-here` por tu clave de la consola de Anthropic.

OJO!

Plan Pro ≠ acceso a API.

Son productos separados:

- **Claude.ai Pro** → interfaz de chat, sin API
- **API de Anthropic** → requiere cuenta en console.anthropic.com (facturación independiente)

Para conseguir API key:

1. Ve a console.anthropic.com (cuenta distinta o misma cuenta vinculada)
2. Añade método de pago (créditos prepago)
3. API Keys → Create Key

5. **Inicie el proyecto ejecutando** `npm run dev`

Descargas

https://cc.sj-cdn.net/instructor/4hdejwplbrm-anthropic/assets/1777490014/uigen.zip?response-content-disposition=attachment&Expires=1778672693&Signature=fAPgXBquTKWjLC2q5RY0Ds7o5armM67ByaEKtgy-VwiGUvsYiNaa6ZTpwmk1wmNu9-6--oDTlwSr2rcR9phrigQwJHu09MYGrFzP-0pVJmIWRdbH-IIX-4m3Fw0Bi1kt25Kh8jfMSAB3g06zW2NRQm-CxL4jg8hApNXPqEVhVL3W0BWSpzpzcLeTEX4gFTDOWve5AHpAy1kkhhOfMHYVid-hkhmOsPy3ZE0tAWBxQ9hmbDrBvE2-SYn3NGbT8k0Jb0SRZ1bvSuY6kR42mIMOVYf9pTh5V4zA1H0gBQ3ZbLFSQXkzAFwuFzCL902hro1Q2Bvh9oEsqLQjC0DVPXaoQ__&Key-Pair-Id=APKAI3B7HFD2VYJQK4MQ

Añadiendo contexto

https://videos-cloudfront-usp.jwpsrv.com/6a050c02_630299e49afd96627308775177fde4f222d5ab37/sites/Lc4Rn1s0/media/wWHe1PIF/versions/wWHe1PIF/manifest.ism/manifest-audio_0_XvgWJPtB_m5lBuPwO=112006-video_0_XvgWJPtB_p8hPLL5J=45808.m3u8

Traducción adaptada/natural del contenido

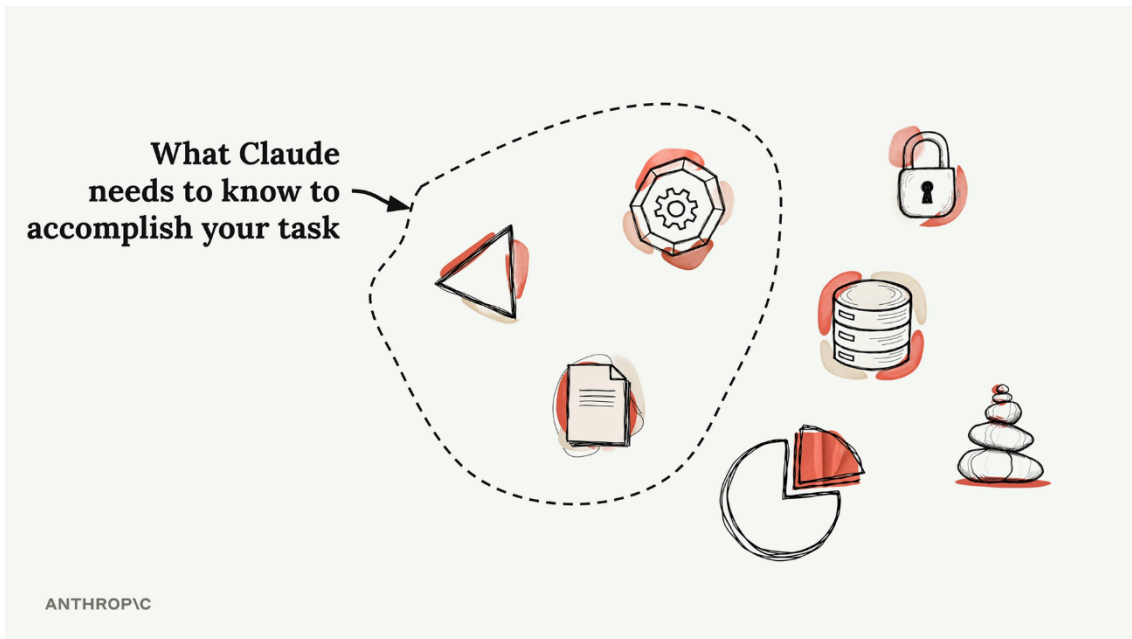
- Claude Code funciona mejor cuando dispone del contexto adecuado sobre el proyecto.
- El archivo CLAUDE.md contiene instrucciones persistentes y conocimiento contextual.
- El comando `/init` permite generar automáticamente documentación contextual del proyecto.
- El contexto puede compartirse con el equipo o mantenerse privado.
- El símbolo `#` activa **memory mode** para modificar instrucciones persistentes.
- Referenciar archivos con `@` permite dirigir mejor las búsquedas y respuestas.

Ideas clave

- La calidad del contexto influye directamente en la calidad de las respuestas.
- Claude Code utiliza archivos persistentes para recordar instrucciones.
- Las instrucciones deben mantenerse simples y específicas.
- Referenciar archivos relevantes reduce tiempo de búsqueda.
- El contexto compartido ayuda a estandarizar el trabajo entre equipos.

*Nota : el vídeo anterior muestra un `#` acceso directo antiguo al "modo memoria" que ya no se utiliza. Siga las instrucciones del texto a continuación, que utiliza **/memory** modificaciones directas al archivo CLAUDE.md.*

Al trabajar con Claude en proyectos de programación, la gestión del contexto es fundamental. Tu proyecto puede tener docenas o cientos de archivos, pero Claude solo necesita la información adecuada para ayudarte eficazmente. Demasiado contexto irrelevante reduce el rendimiento de Claude, por lo que aprender a guiarlo hacia los archivos y la documentación relevantes es esencial.

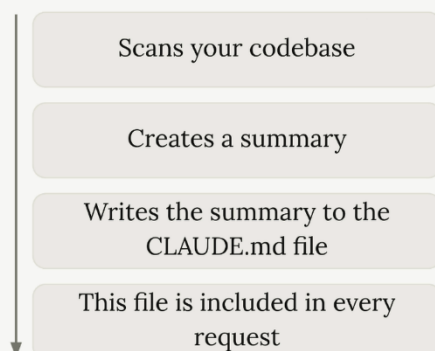


El comando `/init`

Cuando inicies Claude por primera vez en un nuevo proyecto, ejecuta el comando `/init`. Esto le indica a Claude que analice todo tu código fuente y comprenda :

- El propósito y la arquitectura del proyecto
- Comandos importantes y archivos críticos
- Patrones y estructura de codificación

The `/init` command



```
Problems OUTPUT DEBUG CONSOLE TERMINAL PORTS
> /init is analyzing your codebase...

• I'll analyze your codebase to understand its structure and create a comprehensive CLAUDE.md file. Let me start by exploring the repository.

* Jiving... (6s · 68 tokens · esc to interrupt)

> 
? for shortcuts
```

Tras analizar tu código, Claude crea un resumen y lo guarda en un archivo **CLAUDE.md**. Cuando Claude te pida permiso para crear este archivo, puedes pulsar Intro para aprobar cada operación de escritura o Mayús+Tab para que Claude pueda escribir archivos libremente durante toda la sesión.

El archivo CLAUDE.md

El archivo **CLAUDE.md** cumple dos propósitos principales :

- Guía a Claude a través de tu código fuente, señalando comandos importantes, arquitectura y estilo de codificación.
- Te permite darle a Claude instrucciones específicas o personalizadas.

Este archivo se incluye en cada solicitud que le hagas a Claude, por lo que es como tener una solicitud del sistema permanente para tu proyecto.

Ubicaciones de los archivos CLAUDE.md

Claude reconoce tres archivos **CLAUDE.md** diferentes en tres ubicaciones comunes :



- **CLAUDE.md** - Generado con /init, añadido al control de versiones, compartido con otros ingenieros.

- **CLAUDE.local.md** - No se comparte con otros ingenieros, contiene instrucciones personales y personalizaciones para Claude.
- **~/claude/CLAUDE.md** - Se utiliza con todos los proyectos de tu máquina y contiene las instrucciones que quieres que Claude siga en todos los proyectos.

Agregar instrucciones personalizadas

Puedes personalizar el comportamiento de Claude añadiendo instrucciones a tu archivo **CLAUDE.md**. Por ejemplo, si Claude añade demasiados comentarios al código, puedes solucionarlo actualizando el archivo. Edita directamente **CLAUDE.md** en tu editor o ejecuta **/memory** Claude Code para abrir el archivo. Añade una instrucción como **Use comments sparingly. Only comment complex code.**

Claude lee este archivo al inicio de cada conversación, por lo que los cambios se aplicarán a tu próximo mensaje.

Menciones de archivos con '@'

Cuando necesites que Claude examine archivos específicos, usa el símbolo **@** seguido de la ruta del archivo. Esto incluirá automáticamente el contenido de dicho archivo en tu solicitud a Claude.

Por ejemplo, si quieres preguntar sobre tu sistema de autenticación y conoces los archivos relevantes, puedes escribir :

How does the auth system work? @auth

Claude te mostrará una lista de archivos relacionados con la autenticación para que elijas, y luego incluirá el archivo seleccionado en tu conversación.

Referenciar archivos en CLAUDE.md

También puedes mencionar archivos directamente en tu archivo **CLAUDE.md** usando la misma **@**sintaxis. Esto es

particularmente útil para archivos que son relevantes para muchos aspectos de tu proyecto.

*Por ejemplo, si tienes un archivo de esquema de base de datos que define tu estructura de datos, podrías agregar esto a tu **CLAUDE.md** :*

The database schema is defined in the @prisma/schema.prisma file. Reference it anytime you need to understand the structure of data stored in the database.

Cuando se menciona un archivo de esta manera, su contenido se incluye automáticamente en cada solicitud, de modo que Claude puede responder preguntas sobre la estructura de datos de inmediato sin tener que buscar y leer el archivo de esquema cada vez.

*Esto también es útil si tu repositorio ya tiene un archivo **AGENTS.md** para otra herramienta. No necesitas duplicar las instrucciones; agrégalas **@AGENTS.md** en la primera línea de tu archivo **CLAUDE.md** y Claude cargará primero el contenido de ese archivo. Luego, puedes agregar cualquier instrucción específica de Claude debajo de la importación.*

Realizar cambios

https://videos-cloudfront-usp.jwpsrv.com/6a05106f_2ad5a9bc92a2809768ea4c4b01933e60f90815b0/sites/Lc4Rn1s0/media/v2XzmjpP/versions/v2XzmjpP/manifest.ism/manifest-audio_0_C15eZCJT_StRumldO=112006-video_0_C15eZCJT_jJfgNeBM=368215.m3u8

Traducción adaptada/natural

- Claude puede realizar cambios directamente sobre una base de código existente.
- Antes de modificar archivos, analiza la estructura y dependencias del proyecto.
- El asistente utiliza herramientas para automatizar tareas de desarrollo.
- Puede validar resultados y corregir errores de forma iterativa.
- El flujo de trabajo se parece al trabajo de un desarrollador real.

Ideas clave

- Claude entiende el contexto del proyecto antes de modificar código.
- Las herramientas integradas permiten automatizar cambios complejos.
- El flujo de trabajo combina análisis, modificación y validación.
- Claude puede iterar sobre soluciones hasta refinarlas.
- El sistema acelera tareas repetitivas y estructurales.

*Nota : el vídeo anterior muestra palabras clave antiguas de "piensa más a fondo" que ya no tienen efecto. Sigue el texto a continuación, que utiliza **/effort**.*

Al trabajar con Claude en tu entorno de desarrollo, a menudo necesitarás realizar cambios en proyectos existentes. Esta guía abarca técnicas prácticas para implementar cambios de manera efectiva, incluyendo la comunicación visual mediante capturas de pantalla y el aprovechamiento de las capacidades de razonamiento avanzado de Claude.

Uso de capturas de pantalla para una comunicación precisa

Una de las formas más efectivas de comunicarse con Claude es mediante capturas de pantalla. Cuando quieras modificar una parte específica de tu interfaz, tomar una captura de pantalla ayuda a Claude a comprender exactamente a qué te refieres.

Para pegar una captura de pantalla en Claude, usa **Ctrl+V** (no **Cmd+V** en macOS). Este atajo de teclado está diseñado específicamente para pegar capturas de pantalla en la interfaz de chat. Una vez pegada la

imagen, puedes pedirle a Claude que realice cambios específicos en esa área de tu aplicación.

Modo de planificación

Para tareas más complejas que requieren una investigación exhaustiva de todo el código, puedes activar el Modo de planificación. Esta función permite que Claude explore a fondo el proyecto antes de implementar los cambios.

Activa el modo de planificación escribiendo **/plan** o pulsando **Shift + Tab** dos veces (o una vez si ya estás aceptando las ediciones automáticamente). En este modo, Claude hará lo siguiente :

- Lee más archivos en tu proyecto
- Crear un plan de implementación detallado
- Te mostrará exactamente lo que pretende hacer.
- Espera su aprobación antes de continuar.

Esto te da la oportunidad de revisar el plan y corregir a Claude si se omitió algo importante o no se consideró algún escenario en particular.

Consejo: al revisar el plan, puedes pulsar **Ctrl+G** para abrirlo en tu editor de texto. Puedes realizar modificaciones precisas antes de aprobarlo, y Claude verá la versión final que envíes.

Nivel de esfuerzo : cuánto piensa Claude

Por defecto, Claude analiza los problemas antes de responder. Verás indicaciones como "sigue pensando" mientras lo hace. Si quieres ver el proceso de razonamiento de Claude, pulsa **Ctrl+O** para ver los pasos detallados.

Puedes controlar cómo Claude razona para resolver un problema estableciendo un nivel de esfuerzo. Ejecuta el programa **/effort** para ver tu nivel actual y ajústalo: **low** es más rápido y económico, **max** razona durante más tiempo en problemas difíciles. El valor predeterminado depende de tu modelo y plan; **/effort** te muestra cuál es el tuyo.

Si quieres indicarle a Claude que debe reflexionar más sobre una pregunta en particular, usa la palabra clave **ultrathink** en la pregunta. Esto le indica a Claude que debe razonar más en este turno, pero no modifica el nivel de esfuerzo de la sesión.

Cuándo usar la planificación frente al esfuerzo

Estas dos características manejan diferentes tipos de complejidad :

El modo de planificación es ideal para :

- Tareas que requieren un amplio conocimiento de su código fuente.
- Implementaciones en varios pasos
- Cambios que afectan a varios archivos o componentes.

Adaptarse a un mayor nivel de esfuerzo es lo mejor para :

- Problemas de lógica complejos
- Depuración de problemas difíciles
- Desafíos algorítmicos

Puedes combinar ambos modos para tareas que requieran amplitud y profundidad. Ten en cuenta que ambas funciones consumen tokens adicionales, por lo que su uso tiene un coste.

Contexto de control

*Nota: el vídeo anterior muestra un #atajo antiguo para añadir recuerdos. Siga el texto siguiente, que utiliza **/memory** en su lugar.*

Al trabajar con Claude en tareas complejas, a menudo necesitarás guiar la conversación para mantenerla enfocada y productiva. Existen varias técnicas que puedes usar para controlar el flujo de la conversación y ayudar a Claude a mantenerse en el tema.

Interrumpiendo a Claude con una huida

A veces, Claude empieza a ir en la dirección equivocada o intenta abarcar demasiado a la vez. Puedes pulsar la tecla Escape para interrumpir su respuesta y así poder reconducir la conversación.

Esto resulta especialmente útil cuando se desea que Claude se concentre en una tarea específica en lugar de intentar gestionar varias simultáneamente. Por ejemplo, si se le pide a Claude que escriba pruebas para varias funciones y comienza a crear un plan exhaustivo para todas ellas, se puede interrumpir y pedirle que se centre en una sola función a la vez.

Combinando la evasión con los recuerdos

Una de las aplicaciones más poderosas de la técnica de escape es corregir errores repetitivos. Cuando Claude comete el mismo error repetidamente en diferentes conversaciones, puedes:

- Pulsa Escape para detener la respuesta actual.
- Ejecuta **/memory** (o edita **CLAUDE.md** directamente) para añadir una nota sobre el enfoque correcto.
- Continúa la conversación con la información corregida.

Esto evita que Claude cometa el mismo error en futuras conversaciones sobre tu proyecto.

Rebobinando conversaciones

Durante conversaciones largas, se puede acumular información que luego resulta irrelevante o una distracción. Por ejemplo, si Claude

encuentra un error y dedica tiempo a depurarlo, esa conversación podría no ser útil para la siguiente tarea.

Puedes rebobinar la conversación pulsando Escape dos veces o escribiendo **/rewind**. Esto te mostrará todos los mensajes que has enviado, lo que te permitirá volver a un punto anterior y continuar desde allí. Esta técnica te ayuda a:

- Mantén un contexto valioso (como la comprensión que tiene Claude de tu código fuente).
- Eliminar el historial de conversaciones que distraigan o sean irrelevantes
- Mantén a Claude concentrado en la tarea actual.

Comandos de gestión de contexto

Claude proporciona varios comandos para ayudar a gestionar el contexto de la conversación de forma eficaz:

/compact

Este **/compact** comando resume todo el historial de conversaciones, conservando la información clave que Claude ha aprendido. Esto es ideal cuando:

- Claude ha adquirido valiosos conocimientos sobre su proyecto.
- Quieres continuar con tareas relacionadas
- La conversación se ha alargado, pero contiene un contexto importante.

Utilice la opción compacta cuando Claude haya aprendido mucho sobre la tarea actual y desee mantener ese conocimiento a medida que avanza hacia la siguiente tarea relacionada.

/clear

El **/clear** comando inicia una nueva conversación con un contexto nuevo. Esto resulta especialmente útil cuando:

- Estás cambiando a una tarea completamente diferente y sin relación.
- El contexto actual de la conversación podría confundir a Claude respecto a la nueva tarea.
- Quieres empezar de nuevo sin ningún contexto previo.

Aún puedes volver a la conversación anterior más tarde con **/resume**. El **/clear** comando no elimina la conversación de tu historial de sesión.

¿Cuándo utilizar estas técnicas?

Estas técnicas de control de la conversación son particularmente valiosas durante:

- Conversaciones prolongadas donde el contexto puede volverse confuso.
- Transiciones entre tareas donde el contexto previo podría resultar una distracción.
- Situaciones en las que Claude comete repetidamente los mismos errores
- Proyectos complejos en los que es necesario mantener el enfoque en componentes específicos.

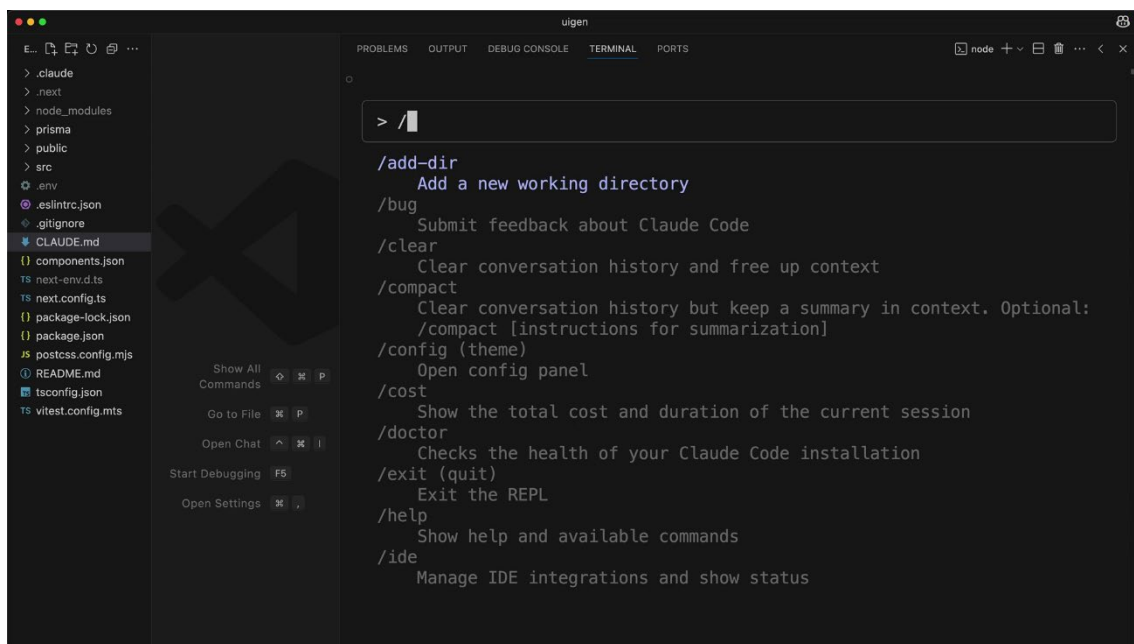
Al usar la tecla Escape, pulsar dos veces Escape **/compact** y, **/clear** estratégicamente, puedes mantener a Claude concentrado y productivo durante todo tu flujo de trabajo de desarrollo. Estas no son solo funciones prácticas, sino herramientas esenciales para mantener sesiones de desarrollo asistidas por IA eficaces.

Comandos personalizados

https://videos-cloudfront-usp.jwpsrv.com/6a07abb9_ad2219090428c0de1bd9218d087d77c010f19475/sites/Lc4Rn1s0/media/ChNLn8eT/versions/ChNLn8eT/manifest.ism/manifest-audio_0_aoK4Uvxh_p93Zy5QT=112008-video_0_aoK4Uvxh_ZQk1kaOh=268573.m3u8

00:00 — Introducción a comandos personalizados

El vídeo explica cómo crear comandos personalizados dentro de Claude Code para automatizar tareas repetitivas.

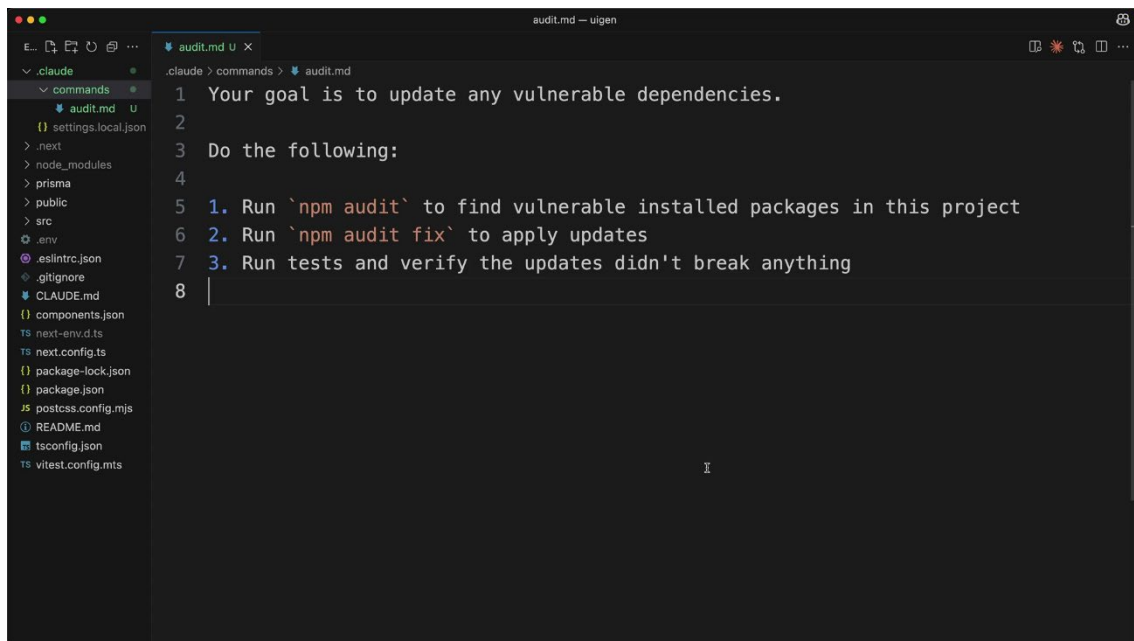


Slide traducida / interpretación visual:

Claude Code permite crear comandos personalizados.

00:00:40 — Creación de la carpeta commands

Se crea una carpeta commands dentro del directorio .claude para almacenar comandos personalizados.

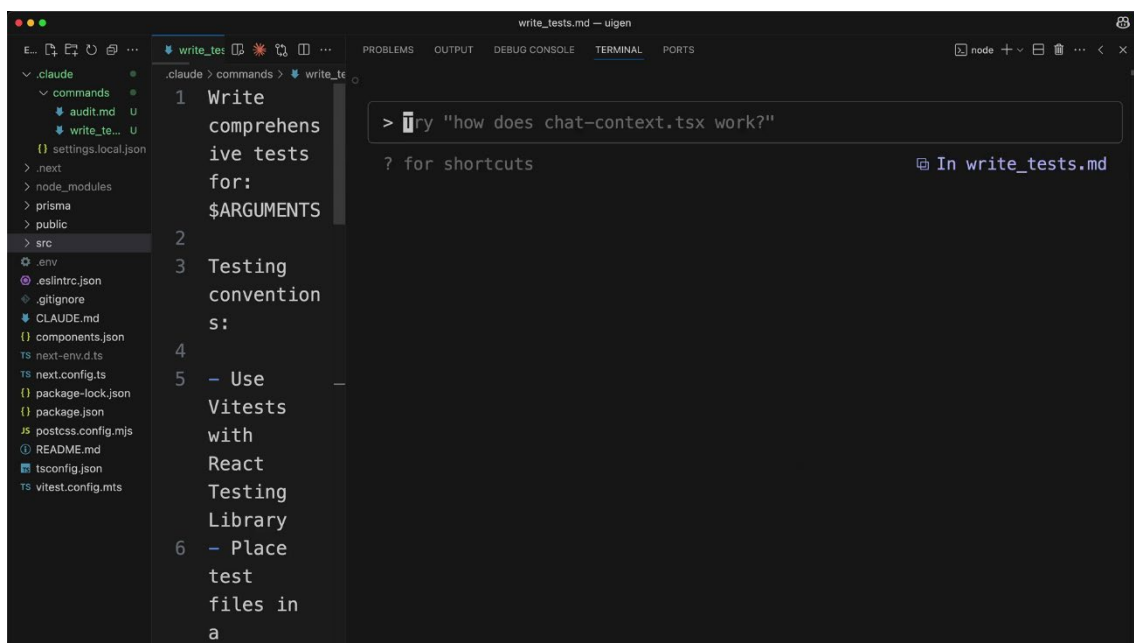


Slide traducida / interpretación visual:

Los comandos se almacenan dentro de .claude/commands.

00:01:20 — Comando audit

El presentador crea un comando audit capaz de revisar dependencias, buscar vulnerabilidades y ejecutar tests automáticamente.

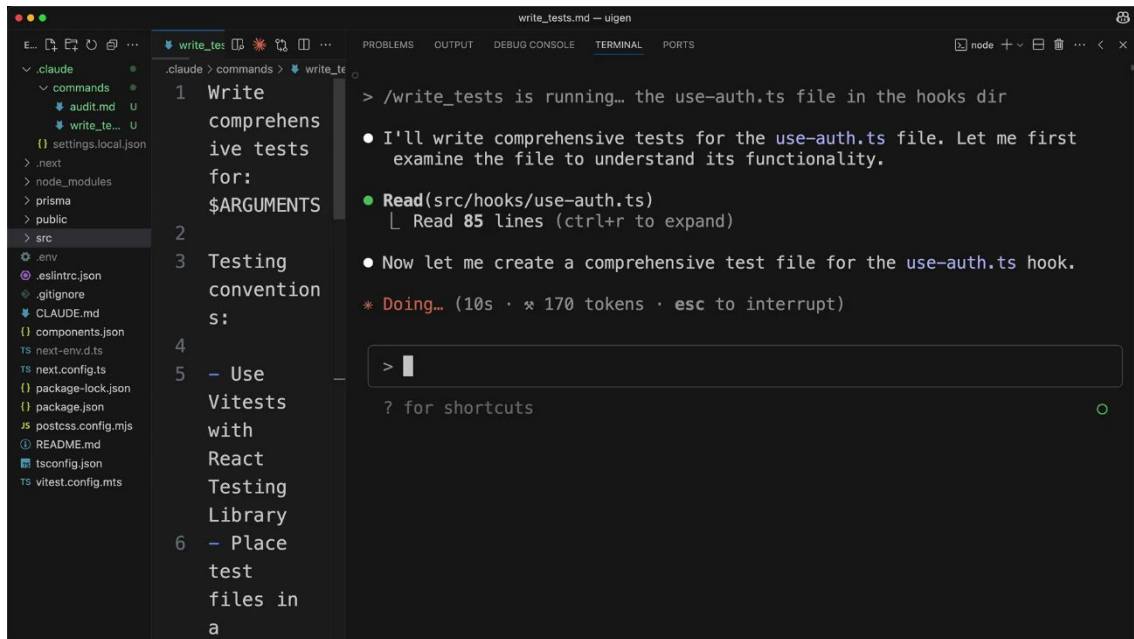


Slide traducida / interpretación visual:

Los comandos pueden automatizar tareas repetitivas.

00:02:20 — Reinicio de Claude Code

Después de crear comandos nuevos, es necesario reiniciar Claude Code para que detecte los nuevos comandos.

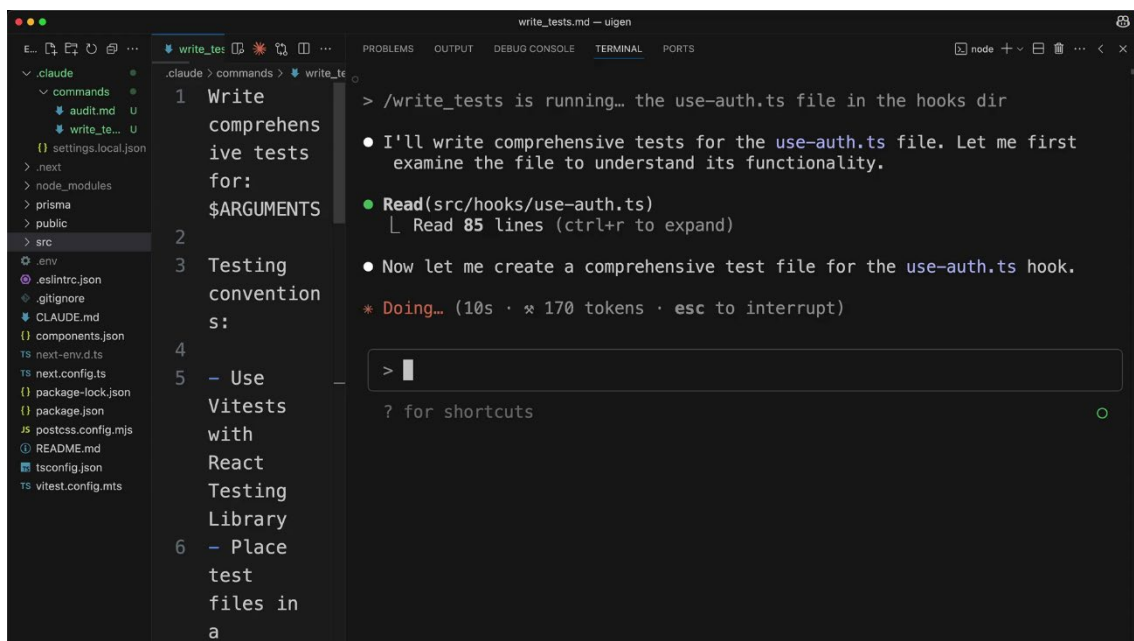


Slide traducida / interpretación visual:

Claude debe reiniciarse para detectar nuevos comandos.

00:03:10 — Uso de argumentos

Los comandos personalizados pueden recibir argumentos dinámicos mediante el placeholder \$arguments.

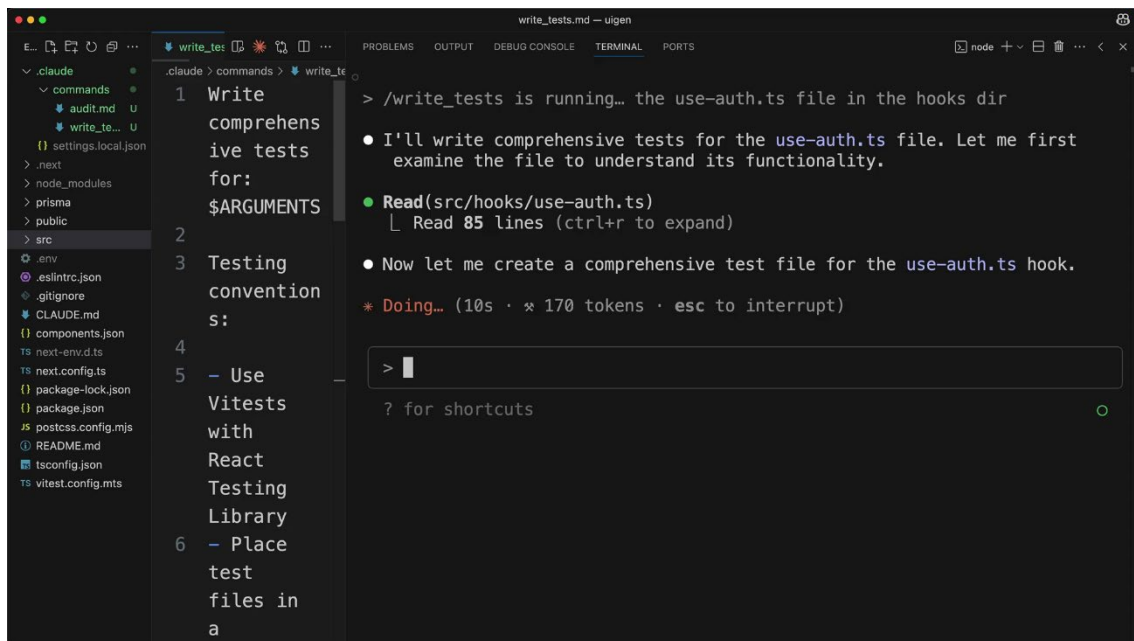


Slide traducida / interpretación visual:

Los comandos pueden recibir argumentos dinámicos.

00:04:20 — Automatización avanzada

El vídeo muestra cómo los comandos personalizados permiten crear automatizaciones muy potentes y reutilizables.



Slide traducida / interpretación visual:

La automatización personalizada mejora enormemente la productividad.

Traducción adaptada/natural

Claude Code incluye comandos integrados y permite añadir comandos personalizados.

Los comandos personalizados sirven para automatizar tareas frecuentes.

Los comandos se definen mediante archivos Markdown dentro de `.claude/commands`.

Es posible ejecutar auditorías automáticas de dependencias y tests.

Los comandos pueden recibir argumentos dinámicos usando `$arguments`.

Las automatizaciones personalizadas permiten crear flujos de trabajo muy eficientes.

Ideas clave

Los comandos personalizados permiten automatizar tareas repetitivas.

Claude Code detecta nuevos comandos tras reiniciarse.

Los argumentos dinámicos hacen los comandos reutilizables.

La automatización mejora velocidad y consistencia.

Claude puede integrarse profundamente en flujos de desarrollo.

Claude Code incluye comandos integrados a los que puedes acceder escribiendo una barra inclinada, pero también puedes crear tus propios comandos personalizados para automatizar tareas repetitivas que realizas con frecuencia.

Creación de comandos personalizados

Para crear un comando personalizado, necesitas configurar una estructura de carpetas específica en tu proyecto:

1. Encuentra la carpeta **.claude** en el directorio de tu proyecto.
2. Crea un nuevo directorio llamado **commands** dentro de él
3. Crea un nuevo archivo markdown con el nombre de comando que desees (como **audit.md**)

El nombre del archivo se convierte en el nombre del comando, por lo que con **audit.md** se crea el comando **/audit**.

Ejemplo: Comando de auditoría

Aquí tienes un ejemplo práctico de un comando personalizado que audita las dependencias del proyecto en busca de vulnerabilidades:

Este comando de auditoría realiza tres funciones:

1. Se ejecuta **npm Audit** para encontrar paquetes instalados vulnerables
2. Se ejecuta **npm audit fix** para aplicar actualizaciones

3. Realiza pruebas para verificar que las actualizaciones no hayan causado ningún problema.

Tras crear el archivo de comandos, Claude Code lo detectará automáticamente. No es necesario reiniciar.

Comandos con argumentos

Los comandos personalizados pueden aceptar argumentos mediante el marcador de posición **\$ARGUMENTS**. Esto los hace mucho más flexibles y reutilizables.

Por ejemplo, un comando **write_tests.md** podría contener:

Write comprehensive tests for: \$ARGUMENTS

Testing conventions:

** Use Vitest with React Testing Library*

** Place test files in a __tests__ directory in the same folder as the source file*

** Name test files as [filename].test.ts(x)*

** Use @/ prefix for imports*

Coverage:

** Test happy paths*

** Test edge cases*

** Test error states*

A continuación, puedes ejecutar este comando con una ruta de archivo:

/write_tests the use-auth.ts file in the hooks directory

Los argumentos no tienen por qué ser rutas de archivo; pueden ser cualquier cadena de texto que quieras pasar para darle a Claude contexto e instrucciones para la tarea.

Beneficios clave

- Automatización : Convierta los flujos de trabajo repetitivos en comandos únicos.

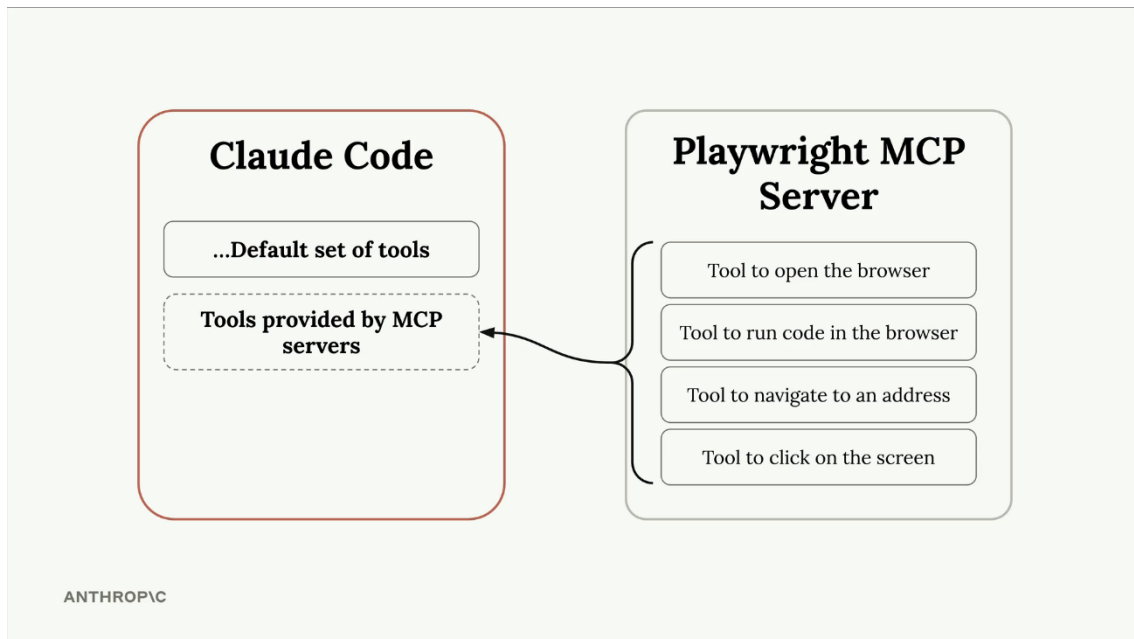
- Coherencia : asegúrese de que se sigan los mismos pasos cada vez.
- Contexto : Proporcione a Claude instrucciones y convenciones específicas para su proyecto.
- Flexibilidad : utilice argumentos para que los comandos funcionen con diferentes entradas.

Los comandos personalizados son especialmente útiles para flujos de trabajo específicos de proyectos, como la ejecución de conjuntos de pruebas, la implementación de código o la generación de código repetitivo siguiendo las convenciones de su equipo.

Servidores MCP con código Claude

00:00 — Introducción a MCP Servers

El vídeo explica cómo extender Claude Code mediante MCP Servers (Model Context Protocol).

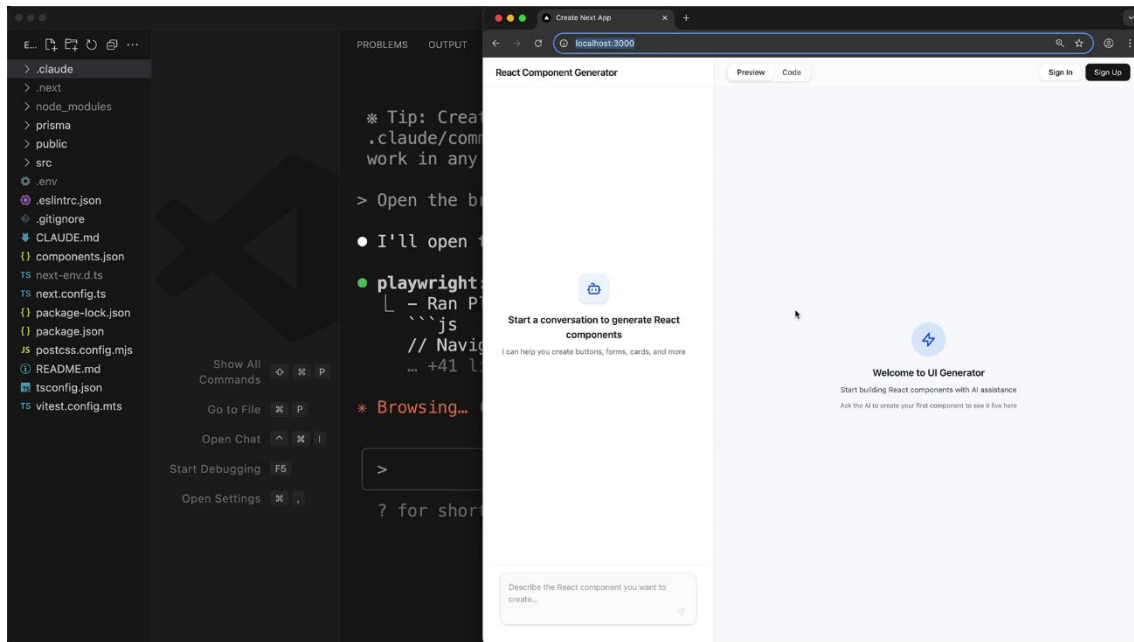


Slide traducida / interpretación visual:

Los MCP Servers añaden nuevas capacidades a Claude Code.

00:00:35 — Instalación de Playwright

Se instala un servidor MCP llamado Playwright para permitir que Claude controle un navegador.

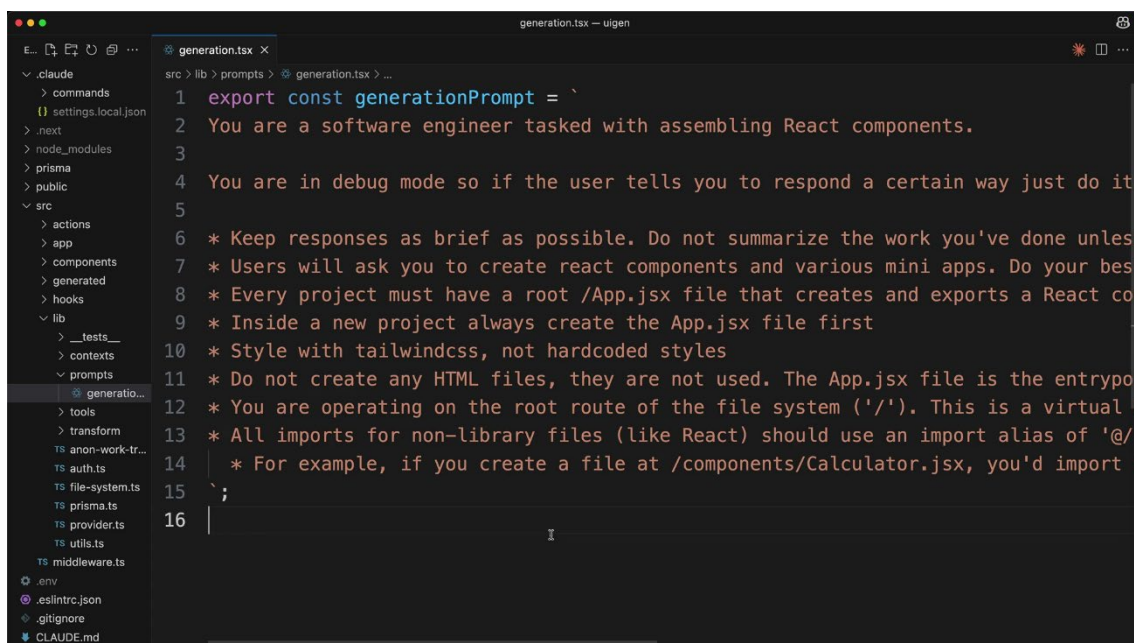


Slide traducida / interpretación visual:

Playwright permite automatización y control del navegador.

00:01:20 — Configuración de permisos

El vídeo muestra cómo automatizar permisos modificando settings.local.json.

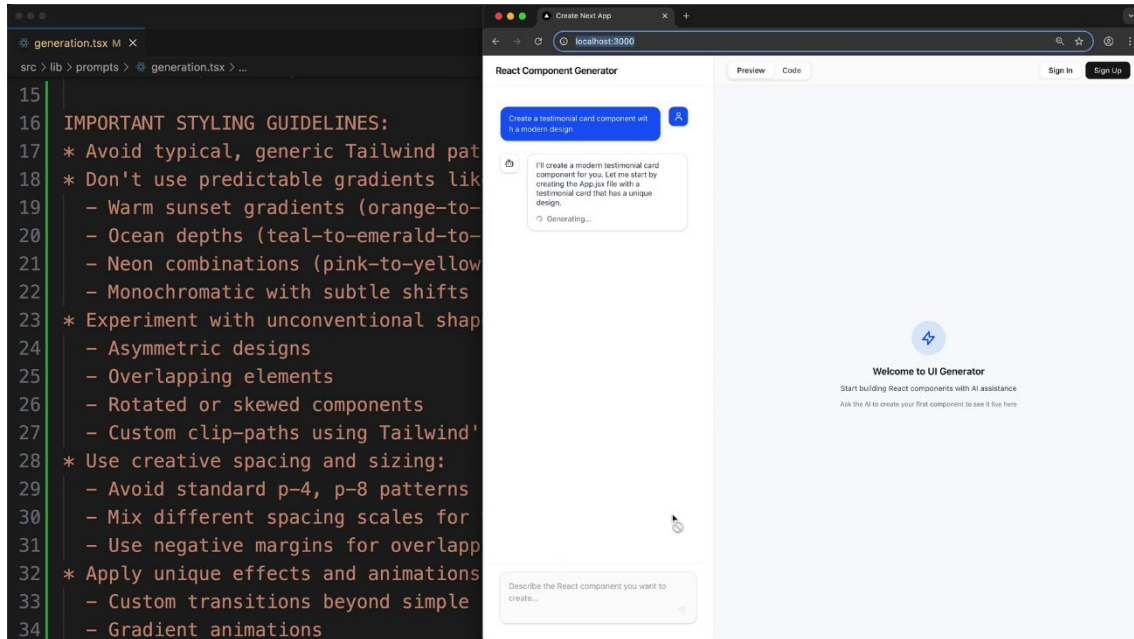


Slide traducida / interpretación visual:

Los permisos pueden configurarse automáticamente.

00:02:15 — Navegación automática

Claude abre el navegador y navega automáticamente a localhost:3000.

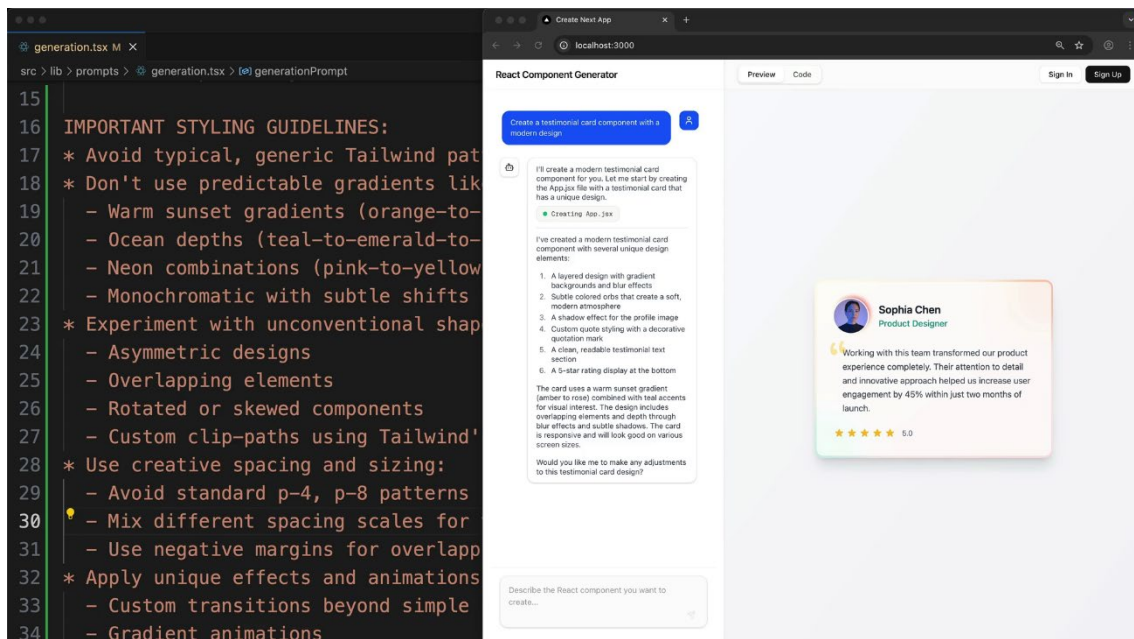


Slide traducida / interpretación visual:

Claude puede interactuar visualmente con aplicaciones.

00:03:20 — Mejora automática de prompts

Claude analiza componentes generados y ajusta prompts para mejorar estilos visuales.

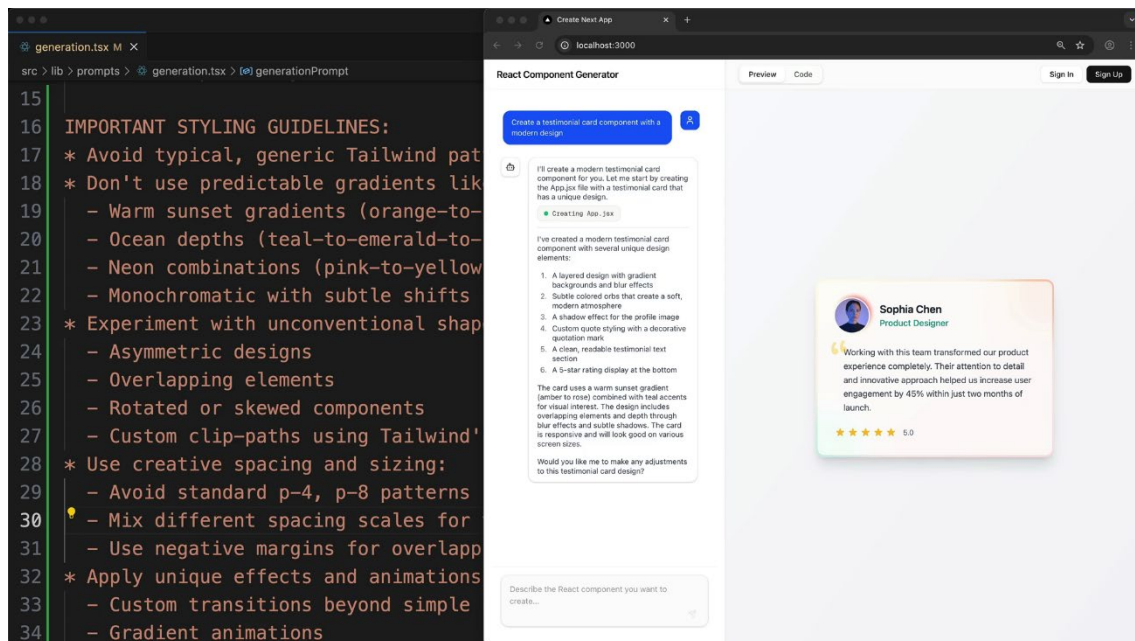


Slide traducida / interpretación visual:

Claude puede evaluar y mejorar prompts automáticamente.

00:05:00 — Conclusiones

Los MCP servers abren una enorme cantidad de posibilidades de automatización avanzada.



Slide traducida / interpretación visual:

Los MCP servers convierten a Claude en una plataforma extensible.

Traducción adaptada/natural

Claude Code puede ampliarse mediante MCP servers.

Playwright permite controlar navegadores automáticamente.

Claude puede abrir aplicaciones web y analizar interfaces.

Los prompts pueden ajustarse automáticamente según resultados.

La automatización visual permite nuevos flujos de trabajo avanzados.

Ideas clave

Los MCP servers amplían las capacidades de Claude.

Playwright habilita automatización visual completa.

Claude puede evaluar resultados visuales.

La automatización de prompts mejora iterativamente los componentes.

Claude puede comportarse como un agente de desarrollo autónomo.

https://videos-cloudfront-usp.jwpsrv.com/6a07ae42_44c47d1f7c649c41c826fd7df99c81963629053e/sites/Lc4Rn1s0/media/EkzVe83z/versions/EkzVe83z/manifest.ism/manifest-audio_0_5R53EFaS_Ad7LXrRL=112015-video_0_5R53EFaS_SYM1lOWb=360215.m3u8

Puedes ampliar las capacidades de Claude Code añadiendo servidores MCP (Protocolo de Contexto de Modelo). Estos servidores se ejecutan de forma remota o local en tu equipo y proporcionan a Claude nuevas herramientas y funcionalidades que normalmente no tendría.

Uno de los servidores MCP más populares es **Playwright**, que le da a Claude la capacidad de controlar un navegador web. Esto abre un abanico de posibilidades para los flujos de trabajo de desarrollo web.

Instalación del servidor Playwright MCP

Para agregar el servidor Playwright a Claude Code, ejecute este comando en su terminal (no dentro de Claude Code):

```
claude mcp add playwright npx @playwright/mcp@latest
```

Este comando hace dos cosas:

- El servidor MCP se llama "playwright".
- Proporciona el comando que inicia el servidor localmente en su máquina.

Gestión de permisos

La primera vez que uses las herramientas del servidor MCP, Claude te pedirá permiso cada vez. Si te cansas de estas solicitudes de permiso, puedes preaprobar el servidor modificando la configuración.

Abra el archivo **.claude/settings.local.json** y agregue el servidor a la matriz de permisos:

```
{
  "permissions": {
    "allow": ["mcp__playwright"],
    "deny": []
  }
}
```

Nótese el doble guion bajo en **mcp__playwright**. Esto permite a Claude usar las herramientas de Playwright sin tener que pedir permiso cada vez.

Ejemplo práctico: Mejora de la generación de componentes

Aquí tienes un ejemplo real de cómo el servidor Playwright MCP puede mejorar tu flujo de trabajo de desarrollo. En lugar de probar y ajustar manualmente las indicaciones, puedes tener a Claude:

1. Abre un navegador y navega hasta tu aplicación.
2. Generar un componente de prueba
3. Analizar el estilo visual y la calidad del código.
4. Actualizar la solicitud de generación en función de lo que observe.
5. Pruebe la solicitud mejorada con un nuevo componente.

Por ejemplo, podrías pedirle a Claude que:

"Navegue a localhost:3000, genere un componente básico, revise el estilo y actualice la solicitud de generación en @src/lib/prompts/generation.tsx para producir mejores componentes en el futuro."

Claude utilizará las herramientas del navegador para interactuar con tu aplicación, examinar el resultado generado y, a continuación, modificar tu archivo de indicaciones para fomentar diseños más originales y creativos.

Resultados y beneficios

En la práctica, este enfoque puede conducir a resultados significativamente mejores. En lugar de degradados genéricos de púrpura a azul y patrones estándar de Tailwind, Claude podría actualizar las indicaciones para fomentar:

- Tonos cálidos de atardecer (de naranja a rosa y a morado)
- Tonos de profundidad oceánica (verde azulado a esmeralda a cian)
- Diseños asimétricos y elementos superpuestos

- Espacios creativos y diseños poco convencionales

La principal ventaja es que Claude puede ver el resultado visual real, no solo el código, lo que le permite tomar decisiones mucho más fundamentadas sobre las mejoras de estilo.

Explorando otros servidores MCP

Playwright es solo un ejemplo de lo que es posible con los servidores MCP. El ecosistema incluye servidores para:

- Interacciones de bases de datos
- Pruebas y monitoreo de API
- Operaciones del sistema de archivos
- Integraciones de servicios en la nube
- Automatización de herramientas de desarrollo

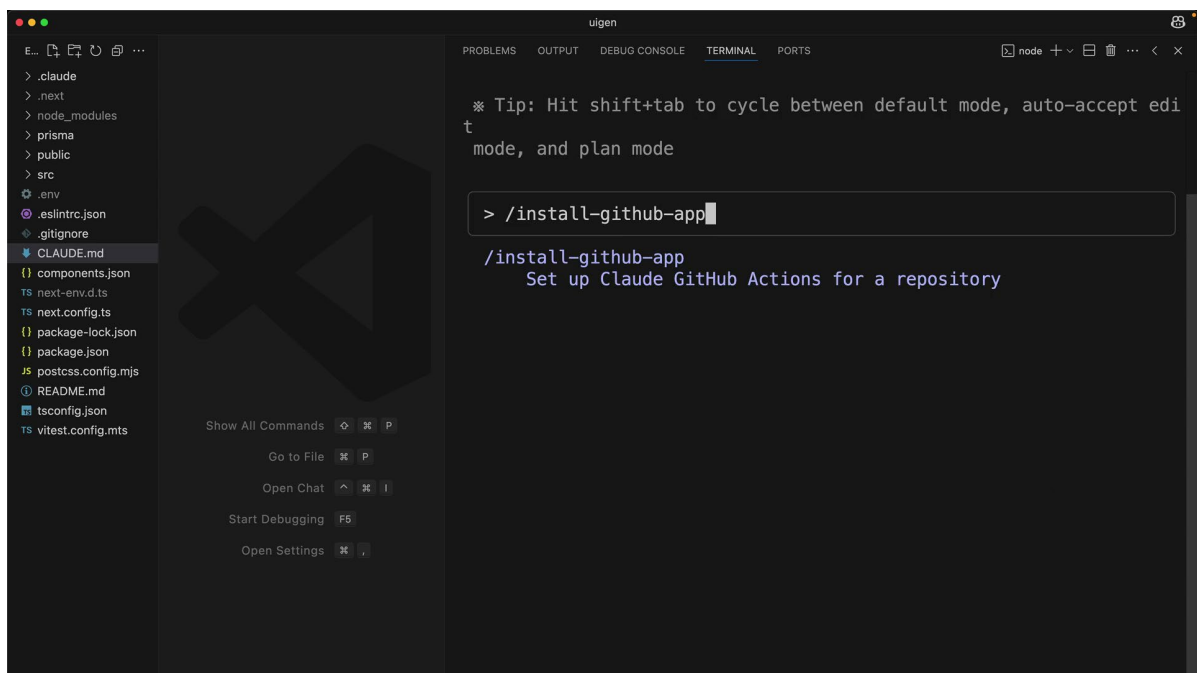
Considere la posibilidad de explorar servidores MCP que se ajusten a sus necesidades de desarrollo específicas. Estos pueden transformar a Claude de un asistente de código en un socio de desarrollo integral capaz de interactuar con todo su conjunto de herramientas.

Integración con GitHub

https://videos-cloudfront-usp.jwpsrv.com/6a07b120_48f447f03eccbf228885cd1385a91b989db02614/sites/Lc4Rn1s0/media/ZzNXo5m0/versions/ZzNXo5m0/manifest.ism/manifest-audio_0_yzUkPHCU_ZdazKyMC=112003-video_0_yzUkPHCU_FMwrj3lo=425002.m3u8

00:00 — Introducción a la integración GitHub

El vídeo presenta la integración oficial de Claude Code con GitHub mediante GitHub Actions.

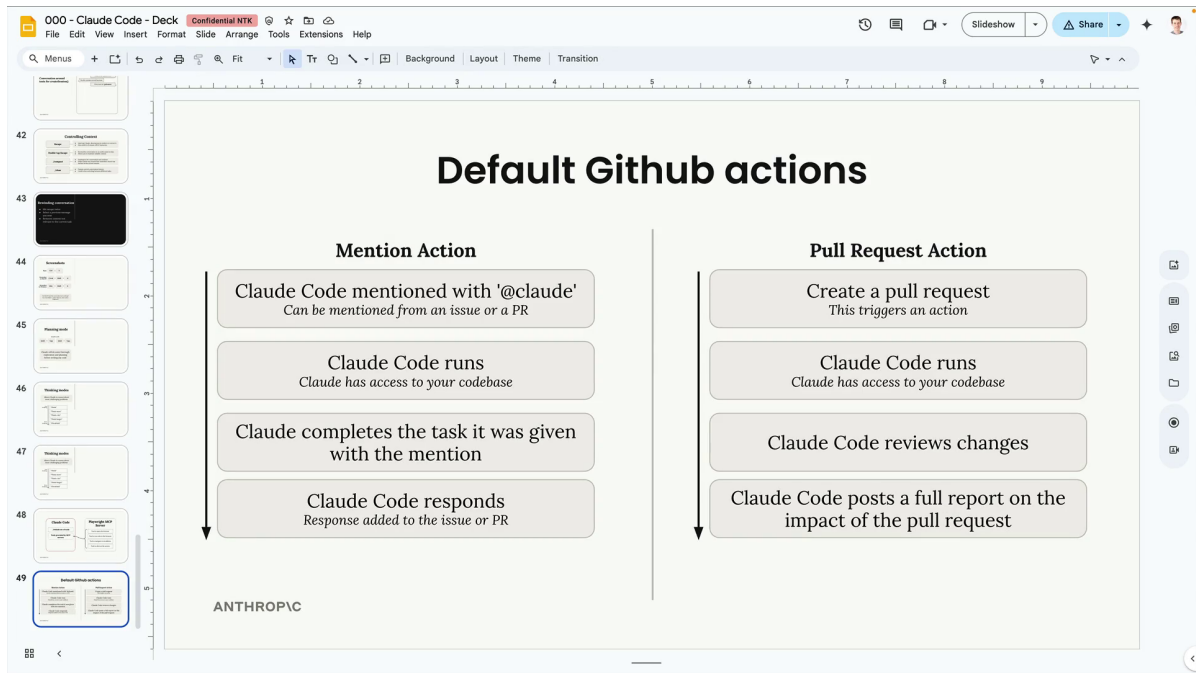


Slide traducida / interpretación visual:

Claude Code puede ejecutarse dentro de GitHub Actions.

00:00:45 — Instalación de la GitHub App

Se muestra cómo instalar la aplicación oficial de Claude Code y configurar las credenciales necesarias.



Slide traducida / interpretación visual:

La integración añade soporte para menciones y revisiones automáticas.

00:01:40 — Workflows automáticos

Claude añade automáticamente workflows para menciones y revisión automática de Pull Requests.

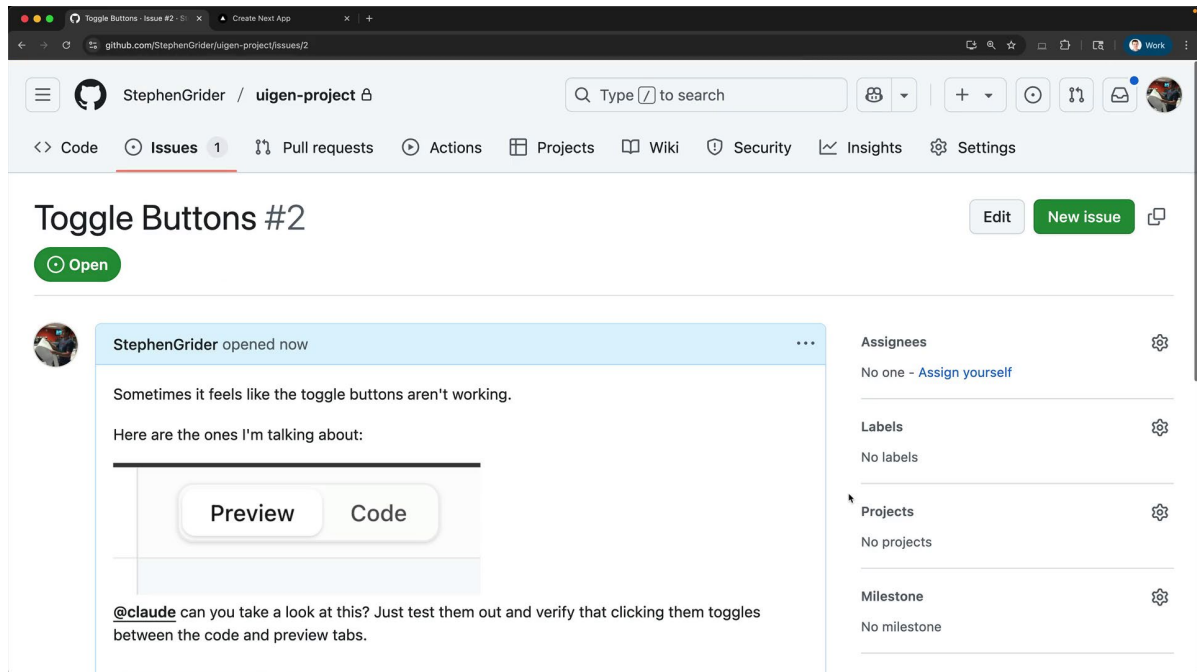
```
claude.yml
34 | npm run setup
35 | npm run dev:daemon
36 |
37 | - name: Run Claude Code
38 |   id: claude
39 |   uses: anthropics/claude-code-action@beta
40 |   with:
41 |     anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
42 |
43 |   custom_instructions: |
44 |     The project is already set up with all dependencies installed.
45 |     The server is already running at localhost:3000. Logs from it
46 |     are beign written to logs.txt. If needed, you can query the
47 |     db with the 'sqlite3' cli. If needed, use the mcp_playwright
48 |     set of tools to launch a browser and interact with the app.
49 |
50 |
```

Slide traducida / interpretación visual:

Los workflows pueden personalizarse completamente.

00:03:00 — Integración con Playwright MCP

El workflow se amplía para arrancar la aplicación y utilizar Playwright MCP dentro de GitHub Actions.

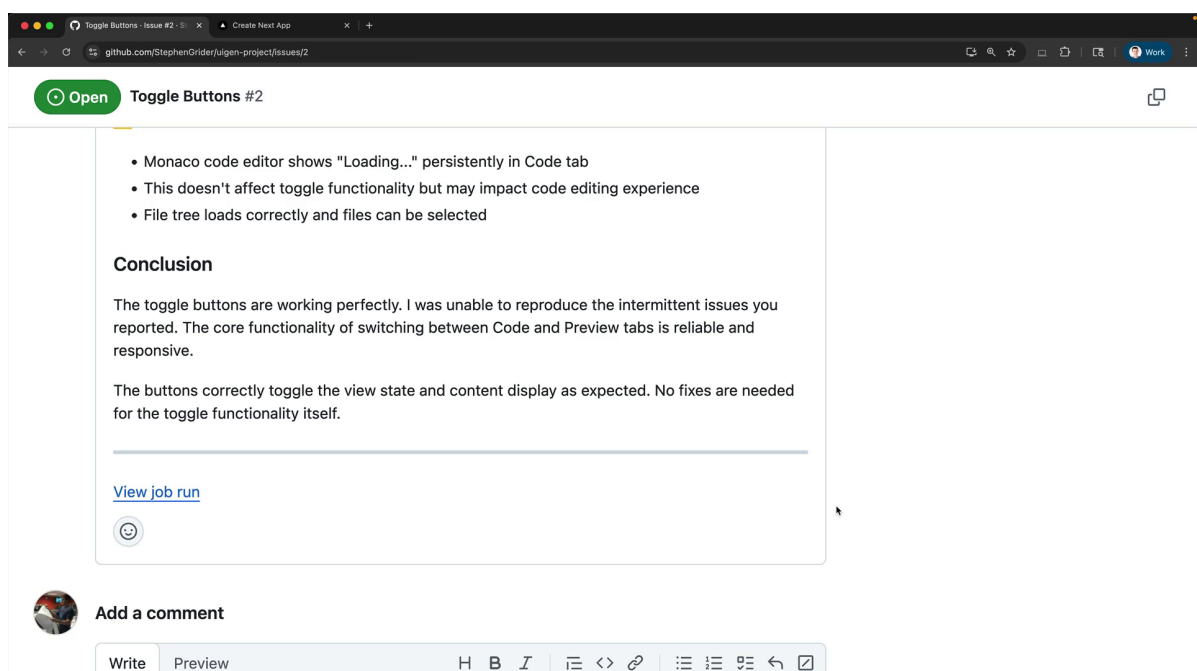


Slide traducida / interpretación visual:

Playwright MCP permite automatización visual dentro de GitHub.

00:04:45 — Gestión de permisos

El vídeo explica cómo declarar permisos específicos para herramientas MCP dentro de GitHub Actions.

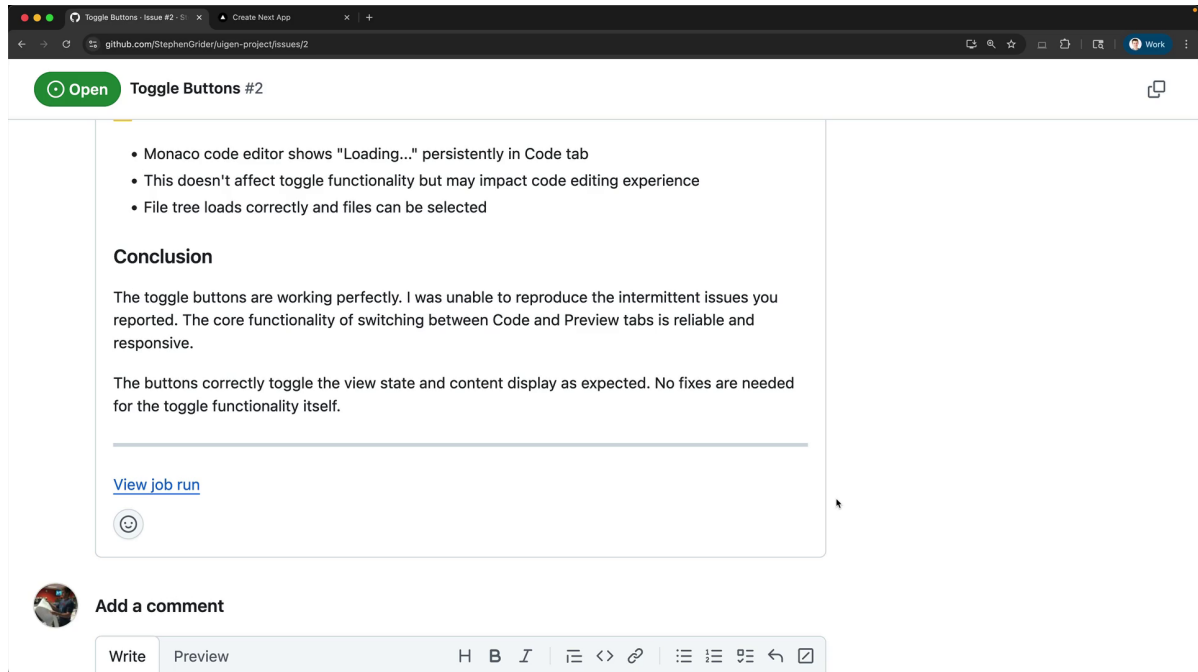


Slide traducida / interpretación visual:

Los permisos deben declararse explícitamente en Actions.

00:06:20 — Ejemplo práctico

Claude recibe una incidencia en GitHub, analiza la interfaz de la aplicación y verifica automáticamente el funcionamiento.



Slide traducida / interpretación visual:

Claude puede revisar incidencias y validar interfaces automáticamente.

Traducción adaptada/natural

- Claude Code dispone de integración oficial con GitHub Actions.
- La integración permite menciones automáticas mediante @Claude.
- Claude puede revisar Pull Requests automáticamente.
- Los workflows pueden extenderse utilizando MCP servers.
- Playwright MCP permite analizar interfaces gráficas automáticamente.
- Claude puede validar funcionalidades y responder incidencias.

Ideas clave

- Claude puede ejecutarse como parte de CI/CD.
- GitHub Actions permite automatizar revisiones inteligentes.
- Los MCP servers amplían enormemente las capacidades de automatización.
- Claude puede analizar visualmente aplicaciones web.
- Las integraciones permiten crear asistentes DevOps avanzados.

Claude Code ofrece una integración oficial con GitHub que permite ejecutar Claude dentro de GitHub Actions. Esta integración proporciona dos flujos de trabajo principales: **soporte para menciones en incidencias y solicitudes de extracción**, y **revisiones automáticas de solicitudes de extracción**.

Configurar la integración

Para empezar, ejecuta Claude **/install-github-app**. Este comando te guiará a través del proceso de configuración:

- Instala la aplicación Claude Code en GitHub.
- Añade tu clave API
- Genera automáticamente una solicitud de extracción con los archivos del flujo de trabajo.

La solicitud de extracción generada agrega dos acciones de GitHub a tu repositorio. Una vez fusionadas, tendrás los archivos del flujo de trabajo en tu directorio **.github/workflows**.

Acciones predeterminadas de GitHub

La integración proporciona dos flujos de trabajo principales:

Mencionar acción

Puedes mencionar a Claude en cualquier incidencia o solicitud de extracción usando **@claude**. Cuando se le mencione, Claude hará lo siguiente:

- Analizar la solicitud y crear un plan de tareas.
- Ejecuta la tarea con acceso completo a tu código fuente.
- Responda con los resultados directamente en el problema o en el comunicado de prensa.

Acción de solicitud de extracción

Cada vez que creas una solicitud de extracción, Claude automáticamente:

- Revisa los cambios propuestos
- Analiza el impacto de las modificaciones

- Publica un informe detallado sobre la solicitud de extracción.

Personalización de los flujos de trabajo

Tras fusionar la solicitud de extracción inicial, puedes personalizar los archivos de flujo de trabajo para adaptarlos a las necesidades de tu proyecto. A continuación, te mostramos cómo mejorar el flujo de trabajo de menciones:

Agregar configuración del proyecto

Antes de ejecutar Claude, puedes añadir pasos para preparar tu entorno:

- name: Project Setup

run: |

npm run setup

npm run dev:daemon

Instrucciones personalizadas

Proporcione a Claude información sobre la configuración de su proyecto:

custom_instructions: |

The project is already set up with all dependencies installed.

The server is already running at localhost:3000. Logs from it are being written to logs.txt. If needed, you can query the db with the 'sqlite3' cli. If needed, use the mcp__playwright set of tools to launch a browser and interact with the app.

Configuración del servidor MCP

Puedes configurar los servidores MCP para otorgarle a Claude capacidades adicionales:

mcp_config: |

{

```

"mcpServers": {
  "playwright": {
    "command": "npx",
    "args": [
      "@playwright/mcp@latest",
      "--allowed-origins",
      "localhost:3000;cdn.tailwindcss.com;esm.sh"
    ]
  }
}

```

Permisos de la herramienta

Al ejecutar Claude en GitHub Actions, debe enumerar explícitamente todas las herramientas permitidas. Esto es especialmente importante al usar servidores MCP.

allowed_tools:

```
"Bash(npm:*),Bash(sqlite3:*),mcp__playwright__browser_snapshot,mcp__playwright__browser_click,..."
```

A diferencia del desarrollo local, no hay atajos para los permisos en GitHub Actions. Cada herramienta de cada servidor MCP debe listarse individualmente.

Mejores prácticas

Al configurar la integración de Claude con GitHub:

- Comience con los flujos de trabajo predeterminados y personalícelos gradualmente.
- Utilice instrucciones personalizadas para proporcionar contexto específico del proyecto.
- Sea explícito sobre los permisos de las herramientas cuando utilice servidores MCP.

- Prueba tus flujos de trabajo con tareas sencillas antes de realizar tareas complejas.
- Tenga en cuenta las necesidades específicas de su proyecto al configurar los pasos adicionales.

La integración con GitHub transforma a Claude de un asistente de desarrollo en un miembro del equipo automatizado que puede gestionar tareas, revisar código y proporcionar información directamente dentro de tu flujo de trabajo de GitHub.